

TEORÍA DE ALGORITMOS  
(75.29) CURSO ECHEVERRÍA

## Trabajo Práctico 2



22 de junio de 2026

Juan Francisco Cuevas  
107963

Victor Zacarías  
107080

Joaquín Hernandez  
110470

Dante Finci  
108456

## 1. Problema 1: Programación Lineal

### 1.1. Descripción del problema

Concesiones Argentina 2000 SRL tiene la concesión de espacios publicitarios en paradas de colectivos de un municipio. En total dispone de 200 paradas, y recibe distintas ofertas de clientes interesados en ocupar cierta cantidad de ellas a cambio de un monto determinado.

El objetivo del problema es decidir qué ofertas aceptar para maximizar el beneficio total obtenido por la empresa, respetando la cantidad máxima de paradas disponibles y las condiciones particulares impuestas por el enunciado.

Las ofertas recibidas son las siguientes:

| Cliente | Opción   | Paradas requeridas | Beneficio ofrecido |
|---------|----------|--------------------|--------------------|
| A       | Única    | 30                 | USD 50.000         |
| B       | Opción 1 | 80                 | USD 100.000        |
| B       | Opción 2 | 120                | USD 120.000        |
| C       | Única    | 75                 | USD 100.000        |
| D       | Única    | 50                 | USD 80.000         |
| E       | Única    | 2                  | USD 5.000          |
| F       | Única    | 20                 | USD 40.000         |
| G       | Única    | 100                | USD 90.000         |

Cuadro 1: Ofertas disponibles para concesionar espacios publicitarios.

Además, se debe considerar que los clientes A y D son competidores directos, por lo que no pueden tener publicidad simultáneamente.

### 1.2. Supuestos

Para formular el modelo se consideran los siguientes supuestos:

- Cada oferta se acepta completa o se rechaza completa. Es decir, no se permite aceptar parcialmente una oferta.
- El beneficio asociado a cada cliente es fijo y se obtiene únicamente si la oferta correspondiente es aceptada.
- No se consideran costos adicionales de instalación, mantenimiento ni administración. Por lo tanto, el beneficio total coincide con la suma de los montos ofrecidos por los clientes seleccionados.
- La empresa dispone como máximo de 200 paradas.
- El cliente B ofrece dos opciones alternativas, pero no se pueden aceptar ambas simultáneamente.
- Los clientes A y D no pueden ser seleccionados al mismo tiempo por ser competidores directos.
- Todas las variables de decisión son binarias, ya que cada oferta puede ser aceptada o rechazada.

### 1.3. Variables de decisión

Se definen las siguientes variables binarias:

$$x_A = \begin{cases} 1 & \text{si se acepta la oferta del cliente A} \\ 0 & \text{en caso contrario} \end{cases}$$

$$x_{B80} = \begin{cases} 1 & \text{si se acepta la oferta del cliente B por 80 paradas} \\ 0 & \text{en caso contrario} \end{cases}$$

$$x_{B120} = \begin{cases} 1 & \text{si se acepta la oferta del cliente B por 120 paradas} \\ 0 & \text{en caso contrario} \end{cases}$$

$$x_C, x_D, x_E, x_F, x_G = \begin{cases} 1 & \text{si se acepta la oferta del cliente correspondiente} \\ 0 & \text{en caso contrario} \end{cases}$$

Todas las variables pertenecen al conjunto:

$$x_i \in \{0, 1\}$$

Por lo tanto, el problema se modela como un problema de programación lineal entera binaria.

## 1.4. Modelo de Programación Lineal

### 1.4.1. Función objetivo

Se busca maximizar el beneficio total obtenido por la empresa. La función objetivo queda definida de la siguiente manera:

$$\text{máx } Z = 50000x_A + 100000x_{B80} + 120000x_{B120} + 100000x_C + 80000x_D + 5000x_E + 40000x_F + 90000x_G$$

### 1.4.2. Restricción de cantidad de paradas

La suma total de paradas utilizadas por los clientes seleccionados no puede superar las 200 paradas disponibles:

$$30x_A + 80x_{B80} + 120x_{B120} + 75x_C + 50x_D + 2x_E + 20x_F + 100x_G \leq 200$$

### 1.4.3. Restricción sobre las opciones del cliente B

El cliente B ofrece dos opciones, pero sólo puede seleccionarse una de ellas:

$$x_{B80} + x_{B120} \leq 1$$

### 1.4.4. Restricción de incompatibilidad entre A y D

Como los clientes A y D son competidores directos, no pueden tener publicidad simultáneamente:

$$x_A + x_D \leq 1$$

## 1.5. Implementación en Python

Para resolver el modelo se utilizó Python junto con la biblioteca PuLP. Esta biblioteca permite definir problemas de optimización lineal, declarar variables de decisión, agregar restricciones y resolver el modelo mediante un solver.

El código utilizado fue el siguiente:

```
1 Algoritmo ResolverProblemaProgramacionLineal
2
3     Crear un problema de maximizacion
4
5     Definir las variables binarias:
6         x_A
7         x_B80
8         x_B120
9         x_C
10        x_D
11        x_E
12        x_F
13        x_G
14
15    Definir la funcion objetivo a maximizar:
16        Z = 50000 x_A
17            + 100000 x_B80
18            + 120000 x_B120
19            + 100000 x_C
20            + 80000 x_D
21            + 5000 x_E
22            + 40000 x_F
23            + 90000 x_G
24
25    Agregar restriccion de cantidad maxima de paradas:
26        30 x_A
27        + 80 x_B80
28        + 120 x_B120
29        + 75 x_C
30        + 50 x_D
31        + 2 x_E
32        + 20 x_F
33        + 100 x_G
34        <= 200
35
36    Agregar restriccion sobre el cliente B:
37        x_B80 + x_B120 <= 1
38
39    Agregar restriccion de incompatibilidad entre A y D:
40        x_A + x_D <= 1
41
42    Resolver el modelo utilizando el solver HiGHS
43
44    Mostrar:
45        Estado de la solucion
46        Beneficio total obtenido
47        Variables seleccionadas
48        Cantidad de paradas utilizadas
49        Cantidad de paradas sin utilizar
50
51 FinAlgoritmo
```

Listing 1: Pseudocodigo de resolucion del Problema 1 mediante PuLP y HiGHS

## 1.6. Resultado obtenido

Al ejecutar el programa, el solver encuentra una solución óptima. La salida obtenida es la siguiente:

```
1 Estado: Optimal
```

```
2 Beneficio total: 255000.0
3
4 Variables seleccionadas:
5 A = 1.0
6 B_80 = 1.0
7 B_120 = 0.0
8 C = 1.0
9 D = 0.0
10 E = 1.0
11 F = 0.0
12 G = 0.0
13
14 Paradas utilizadas: 187.0
15 Paradas disponibles sin utilizar: 13.0
```

Listing 2: Salida obtenida por el programa

Por lo tanto, las ofertas seleccionadas son:

| Cliente seleccionado | Paradas utilizadas | Beneficio obtenido |
|----------------------|--------------------|--------------------|
| A                    | 30                 | USD 50.000         |
| B, opción 80 paradas | 80                 | USD 100.000        |
| C                    | 75                 | USD 100.000        |
| E                    | 2                  | USD 5.000          |
| Total                | 187                | USD 255.000        |

Cuadro 2: Solución óptima obtenida.

### 1.7. Verificación de restricciones

Se verifica que la solución cumple con todas las restricciones del problema.

En primer lugar, la cantidad de paradas utilizadas es:

$$30 + 80 + 75 + 2 = 187$$

Como:

$$187 \leq 200$$

la restricción de capacidad se cumple.

En segundo lugar, se selecciona únicamente una opción del cliente B:

$$x_{B80} + x_{B120} = 1 + 0 = 1$$

Por lo tanto, también se cumple la restricción correspondiente al cliente B.

Finalmente, no se seleccionan simultáneamente los clientes A y D:

$$x_A + x_D = 1 + 0 = 1$$

Por lo tanto, se respeta la incompatibilidad entre ambos clientes.

### 1.8. Interpretación de la solución

La solución óptima indica que Concesiones Argentina 2000 SRL debería aceptar las ofertas de los clientes A, B en su opción de 80 paradas, C y E.

Esta combinación utiliza 187 de las 200 paradas disponibles y genera un beneficio total de:

$$Z = USD\ 255,000$$

Quedan sin utilizar:

$$200 - 187 = 13$$

paradas.

Si bien no se ocupan todas las paradas disponibles, esta es la combinación que maximiza el beneficio total respetando las restricciones del problema. No existe otra combinación factible que produzca un beneficio mayor sin superar las 200 paradas disponibles, sin aceptar ambas opciones del cliente B, o sin seleccionar simultáneamente a los clientes A y D.

## 1.9. Conclusión

El problema fue modelado como un problema de programación lineal entera binaria, dado que cada oferta sólo puede aceptarse o rechazarse. A partir del modelo formulado y de la resolución computacional mediante PuLP, se obtuvo que la máxima ganancia posible es de USD 255.000.

La decisión óptima consiste en aceptar las ofertas de los clientes A, B por 80 paradas, C y E, utilizando un total de 187 paradas.

## 2. Problema 2: Backup de antenas

### 2.1. Introduccion

El problema es el siguiente: se tiene una red de  $n$  antenas, todas conectadas entre si, con una matriz  $d = d[1..n, 1..n]$  con las distancias entre todas las antenas. Para ellas se quiere asignar un backup a cada antena, de tal forma que cada antena tenga  $k$  vecinos capaces de hacerle backup pero con un límite  $b$  de estas. El objetivo es encontrar una asignación de backups que cumpla con estas restricciones, o determinar que no existe tal asignación.

### 2.2. Ejemplos

#### 2.2.1. Caso con solución $b = 2$

Supongamos que tenemos 3 antenas en línea recta, con  $k = 1$ ,  $b = 2$  y  $D = 10$ . La matriz de distancias es la siguiente:

$$d = \begin{bmatrix} 0 & 8 & 16 \\ 8 & 0 & 8 \\ 16 & 8 & 0 \end{bmatrix}$$

Proponemos nombrar ‘vecinos’ a las antenas que se encuentran a una distancia menor a  $D$ .

| Antena | Vecinos disponibles | Backup requerido | Opciones     |
|--------|---------------------|------------------|--------------|
| 1      | {2}                 | $k = 1$          | Antena 2     |
| 2      | {1, 3}              | $k = 1$          | Antena 1 o 3 |
| 3      | {2}                 | $k = 1$          | Antena 2     |

Cuadro 3: Vecinos disponibles para cada antena (distancia  $< D = 10$ ). Las antenas 1 y 3 tienen una única opción: elegir antena 2.

El problema surge inmediatamente: las antenas 1 y 3 no tienen opción — ambas *deben* elegir a la antena 2 como backup. Esto significa que la antena 2 aparecería en 2 conjuntos de backup (el de 1 y el de 3), llegando a la restricción  $b = 2$ . Luego, la antena 2 puede elegir entre sus vecinos {1, 3} para hacer backup. Sin embargo, si hubiéramos tenido  $b = 1$ , entonces la antena 2 no podría ser backup de ambas antenas 1 y 3, y no habría solución posible. Veamos las soluciones posibles:

| Antena                                   | Backup asignado | Apariciones como backup      |
|------------------------------------------|-----------------|------------------------------|
| <i>Opción A: Antena 2 elige antena 1</i> |                 |                              |
| 1                                        | 2               | Aparece en: {2} → 1 vez      |
| 2                                        | 1               | Aparece en: {1, 3} → 2 veces |
| 3                                        | 2               | Aparece en: {1, 3} → 2 veces |
| <i>Opción B: Antena 2 elige antena 3</i> |                 |                              |
| 1                                        | 2               | Aparece en: {1, 3} → 2 veces |
| 2                                        | 3               | Aparece en: {2} → 1 vez      |
| 3                                        | 2               | Aparece en: {1, 2} → 2 veces |

Cuadro 4: Ambas opciones son válidas con  $b = 2$ : cada antena aparece en a lo sumo 2 conjuntos de backup.

#### 2.2.2. Caso sin solución: $b = 1$

Consideremos el mismo ejemplo anterior pero con  $b = 1$  en lugar de  $b = 2$ . Las antenas 1 y 3 *deben* elegir a la antena 2 como backup (es la única opción), lo que significa que la antena 2 aparecería en 2 conjuntos de backup. Esto **viola inmediatamente** la restricción  $b = 1$  (máximo 1 aparición):

| Antena                                                                                                                                                                                                            | Intento de asignación | Apariciones |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-------------|
| 1                                                                                                                                                                                                                 | 2 (obligatorio)       | —           |
| 2                                                                                                                                                                                                                 | 1 o 3                 | —           |
| 3                                                                                                                                                                                                                 | 2 (obligatorio)       | —           |
| <b>Análisis del conflicto</b>                                                                                                                                                                                     |                       |             |
| La antena 2 aparecería como backup de las antenas 1 y 3, es decir, 2 apariciones. Pero $b = 1$ exige máximo 1 aparición. No importa lo que elija la antena 2 como su propio backup, ya ha violado la restricción. |                       |             |

Cuadro 5: Con  $b = 1$ : la antena 2 debe ser backup de ambas 1 y 3  $\Rightarrow 2$  apariciones  $> b = 1 \Rightarrow$  **No existe solución.**

## 2.3. Modelado del problema

Para resolver el problema, lo modelamos como un problema de flujo máximo en una red de flujo. La idea es construir una red de flujo tal que encontrar un flujo máximo en esa red corresponda a encontrar una asignación de backups que cumpla con las restricciones del problema.

Nuestra propuesta de modelado es la siguiente:

- Creamos un nodo fuente  $S$  y un nodo sumidero  $T$ .
- Para cada antena  $i$ , creamos dos nodos:  $A_i$  (representando la antena) y  $B_i$  (representando el backup de la antena).
- Conectamos el nodo fuente  $S$  a cada nodo  $A_i$  con una arista de capacidad  $k$  (el tamaño requerido del conjunto de backup).
- Conectamos cada nodo  $B_i$  al nodo sumidero  $T$  con una arista de capacidad  $b$  (el límite de apariciones como backup).
- Para cada par de antenas  $(i, j)$  que son vecinos (es decir,  $d[i][j] < D$ ), conectamos el nodo  $A_i$  al nodo  $B_j$  con una arista de capacidad 1 (representando la posibilidad de que la antena  $i$  elija a la antena  $j$  como backup).
- El objetivo es encontrar un flujo máximo en esta red. Si el flujo máximo es igual a  $n \cdot k$  (es decir, cada antena tiene asignados exactamente  $k$  backups), entonces existe una solución válida. De lo contrario, no existe tal asignación.

## 2.4. Diseño de la solución

### 2.4.1. Adaptación al problema de flujo máximo

La clave del diseño es reconocer que el problema tiene la estructura de una **asignación bipartita con cuotas**: de un lado están las antenas que *demandan* backups, del otro las antenas que *proveen* backups, y el flujo que pasa por la red representa exactamente qué antena respalda a cuál.

La red se construye con  $2n + 2$  nodos organizados en cuatro capas:

1. **Fuente  $S$ :** inyecta  $k$  unidades de flujo hacia cada antena demandante. La capacidad  $k$  de la arista  $S \rightarrow A_i$  fuerza a que cada antena obtenga *exactamente*  $k$  backups (ni más, ni menos, dado que el flujo total debe ser  $n \cdot k$ ).
2. **Nodos  $A_i$  (demanda):** cada antena  $i$  que necesita backup. El flujo que sale de  $A_i$  representa qué antenas conforman su conjunto de backup.
3. **Nodos  $B_j$  (oferta):** cada antena  $j$  en su rol de proveedora de backup. La arista  $A_i \rightarrow B_j$  existe solo si  $d[i][j] < D$ , con capacidad 1 (una antena puede aparecer a lo sumo una vez en el backup de cada otra).



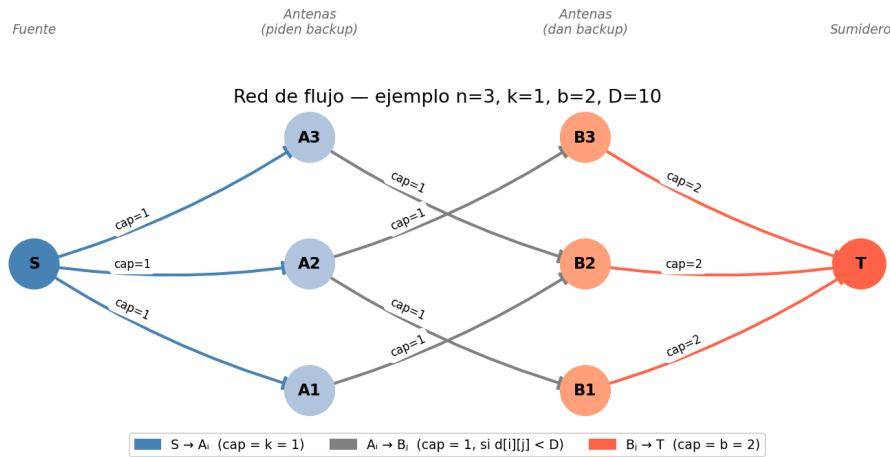


Figura 1: Red de flujo para el ejemplo con  $n = 3, k = 1, b = 2, D = 10$ . Los nodos  $A_i$  piden backup; los nodos  $B_j$  lo proveen. El flujo máximo  $= n \cdot k = 3$  indica que existe solución. Generado con `grafico_red.py`.

4. **Sumidero  $T$ :** absorbe el flujo de todos los  $B_j$ . La capacidad  $b$  de la arista  $B_j \rightarrow T$  limita cuántos conjuntos de backup distintos puede integrar la antena  $j$ .

**Interpretación de la salida.** Una vez ejecutado Edmonds-Karp, la respuesta se lee directamente de la red residual:

- Si **flujo máximo**  $= n \cdot k$ : existe solución. La arista  $A_i \rightarrow B_j$  está saturada (capacidad residual  $= 0$ ) si y solo si  $j$  pertenece al conjunto de backup de  $i$ .
- Si **flujo máximo**  $< n \cdot k$ : no existe solución. El corte mínimo certifica que algún subconjunto de antenas no puede obtener suficientes backups respetando el límite  $b$ .

#### 2.4.2. Pseudocódigo

```

1  funcion OBTENER_CONJUNTO_BACKUP(distancias[1..n][1..n], k, b, D):
2
3      // --- Construccion de la red de flujo ---
4      S = 0
5      T = 2n + 1
6      cap[0..2n+1][0..2n+1] = 0          // matriz de capacidades residuales
7
8      para i = 1..n:
9          cap[S][i] = k                  // S -> A_i : antena i necesita k backups
10
11     para i = 1..n:
12         para j = 1..n:
13             si i != j y distancias[i][j] < D:
14                 cap[i][n+j] = 1        // A_i -> B_j : j puede ser backup de i
15
16     para j = 1..n:
17         cap[n+j][T] = b                // B_j -> T : j aparece en <= b conjuntos
18
19     // --- Flujo maximo (Edmonds-Karp) ---
20     flujo = EDMONDS_KARP(cap, S, T)
21
22     si flujo != n * k:
23         retornar SIN_SOLUCION          // corte minimo < n*k: imposible
24
25     // --- Reconstruccion de los conjuntos ---
26     conjuntos = lista vacia de n listas
    
```

```

27     para i = 1..n:
28         para j = 1..n:
29             si i != j y distancias[i][j] < D y cap[i][n+j] == 0:
30                 conjuntos[i].agregar(j) // arista saturada -> j es backup de i
31
32     retornar conjuntos
33
34
35 funcion EDMONDS_KARP(cap, S, T):
36     flujo_total = 0
37     repetir:
38         padre = BFS(cap, S, T) // camino minimo en aristas en red residual
39         si no existe camino S -> T:
40             terminar
41         // capacidad del cuello de botella
42         cuello = min{ cap[padre[v]][v] : v en camino S -> T }
43         // actualizar red residual
44         para cada arista (u, v) en el camino S -> T:
45             cap[u][v] -= cuello
46             cap[v][u] += cuello
47         flujo_total += cuello
48     retornar flujo_total

```

Listing 3: Pseudocódigo del algoritmo de backup de antenas

**Estructuras de datos.** La red residual se representa como una **matriz de adyacencia**  $\text{cap}[u][v]$  de tamaño  $(2n+2) \times (2n+2)$ , donde  $\text{cap}[u][v]$  almacena la capacidad residual de  $u$  a  $v$ . Esta elección simplifica la actualización de aristas inversas (siempre existe  $\text{cap}[v][u]$ ) a costa de  $O(n^2)$  de memoria, aceptable para los tamaños de instancia esperados.

## 2.5. Complejidad

El algoritmo tiene tres fases bien diferenciadas en el pseudocódigo. Designamos  $V = 2n + 2$  la cantidad de nodos y  $E$  la cantidad de aristas de la red.

**Fase 1 — Construcción de la red** La inicialización de la matriz  $\text{cap}$  requiere  $O(n^2)$  tiempo y espacio (la matriz tiene  $(2n+2)^2$  entradas). El lazo doble de las líneas 11–14 recorre todos los pares  $(i, j)$  con  $i \neq j$  para verificar  $d[i][j] < D$ :

$$\underbrace{O(n)}_{S \rightarrow A_i} + \underbrace{O(n^2)}_{A_i \rightarrow B_j} + \underbrace{O(n)}_{B_j \rightarrow T} = O(n^2)$$

La cantidad de aristas que se agregan a la red es a lo sumo  $E = n + n(n-1) + n = O(n^2)$ .

**Fase 2 — Edmonds-Karp** Cada iteración del ciclo principal ejecuta un BFS sobre la red residual. La red tiene  $V = 2n + 2 = O(n)$  nodos y  $E = O(n^2)$  aristas, por lo que cada BFS cuesta  $O(V + E) = O(n^2)$ .

Para acotar la cantidad de iteraciones usamos que todas las capacidades son enteras, por lo que cada camino aumentante incrementa el flujo en al menos 1. Como el flujo máximo es  $n \cdot k$  (la demanda total de la red), el número de caminos aumentantes es a lo sumo  $n \cdot k$ . En consecuencia, la fase completa toma:

$$O(n \cdot k) \times O(n^2) = O(k \cdot n^3)$$

El límite clásico de Edmonds-Karp es  $O(V \cdot E^2) = O(n^5)$ , pero ese cota no aprovecha que el flujo máximo de *esta* red está acotado por  $n \cdot k$  (usualmente mucho menor que el peor caso genérico).

**Fase 3 — Reconstrucción** El lazo doble recorre los mismos  $O(n^2)$  pares  $(i, j)$  y lee directamente de la matriz residual si la arista está saturada. Costo:  $O(n^2)$ .

**Costo total.** La fase dominante es Edmonds-Karp:

$$O(k \cdot n^3)$$

Para los parámetros típicos del problema ( $k \ll n$ ) esto es efectivamente  $O(n^3)$ , confirmando que el algoritmo es polinomial en el tamaño de la instancia.

## 2.6. Supuestos

Los supuestos sobre los que este algoritmo se basa son:

- Cuando el enunciado dice: "Plantear un algoritmo de complejidad polinomial que encuentre el conjunto de backup de tamaño  $k$  de cada una de las  $n$  antenas", se interpreta que el conjunto de backup de tamaño  $k$  se refiere a que cada antena debe tener por lo menos  $k$  antenas de backup asignadas, no que el grupo de backup de toda la red debe tener tamaño  $k$ .
- La red es conexa, es decir, existe un camino entre cualquier par de antenas.
- Las distancias entre antenas son simétricas, es decir,  $d[i][j] = d[j][i]$  para todas las antenas  $i$  y  $j$ .
- Ninguna antena puede ser su propio backup.
- Las distancias entre antenas no son negativas.
- La matriz de distancias esta completa, incluyendo la diagonal principal (es decir,  $d[i][i] = 0$  para todas las antenas  $i$ ).
- Los parámetros  $k$  y  $b$  son enteros no negativos, y  $D$  es un número real positivo.
- No se considera el posicionamiento en el espacio de las antenas, solo sus distancias mutuas (ignoramos la representacion fisica de las antenas si estan en 2 o 3 dimensiones).
- El valor  $k$  es mucho menor a  $n$ , lo que hace que la fase de Edmonds-Karp sea efectivamente  $O(n^3)$  en la práctica.
- El valor de  $b$  es menor o igual a  $n$ , ya que no tiene sentido que una antena pueda ser backup de más antenas que las que existen en la red.

## 2.7. Evaluaciones y resultados

Se ejecutaron 10 casos de prueba sobre el algoritmo implementado. Los casos fueron generados aleatoriamente con semilla fija (para reproducibilidad) mediante el script `generar_caso.py`, ubicando las antenas en coordenadas 2D uniformes dentro de un área de  $100 \times 100$ , con distancias euclídeas. El caso 1 fue construido manualmente. Las mediciones se realizaron en una misma máquina y corresponden al tiempo de ejecución del algoritmo de flujo máximo (sin contar lectura de archivos).

| Caso                   | Parametros          | Resultado    | Tiempo (ms) |
|------------------------|---------------------|--------------|-------------|
| caso1                  | n=5 k=2 b=3 D=7     | Solucion     | 0.09        |
| caso_b_igual_k         | n=6 k=2 b=2 D=40.0  | Sin solucion | 0.08        |
| caso_b_menor_k         | n=8 k=3 b=2 D=40.0  | Sin solucion | 0.16        |
| caso_d_chico           | n=10 k=3 b=4 D=12.0 | Sin solucion | 0.09        |
| caso_d_amplio          | n=10 k=3 b=4 D=45.0 | Sin solucion | 0.35        |
| caso_ajustado          | n=15 k=2 b=3 D=35.0 | Sin solucion | 0.57        |
| caso_matching_perfecto | n=20 k=1 b=1 D=30.0 | Sin solucion | 0.64        |
| caso_mediano           | n=30 k=4 b=5 D=50.0 | Solucion     | 6.64        |
| caso_grande            | n=50 k=3 b=4 D=30.0 | Solucion     | 19.31       |
| caso_grande_imposible  | n=50 k=3 b=4 D=15.0 | Sin solucion | 18.08       |

14

Listing 4: Resultados de ejecución sobre los 10 casos de prueba (generado por `ejecutar_casos.py`)

### Observaciones.

- **caso\_b\_menor\_k:** con  $b = 2 < k = 3$ , la capacidad total de la red ( $n \cdot b = 16$ ) es menor a la demanda total ( $n \cdot k = 24$ ), por lo que es imposible estructuralmente, independientemente de las distancias.
- **caso\_d\_chico vs. caso\_d\_amplio:** mismo layout geométrico (misma semilla), distinto  $D$ . Ambos resultan sin solución porque con  $n = 10$  antenas dispersas en  $100 \times 100$ ,  $D = 45$  no garantiza 3 vecinos por antena en esa distribución particular.
- **Tiempo de ejecución:** el tiempo crece con  $n$ , consistente con la complejidad  $O(V \cdot E^2)$  de Edmonds-Karp. El caso **caso\_grande\_imposible** ( $n = 50$ ) tarda casi lo mismo que **caso\_grande** porque Edmonds-Karp igualmente explora la red antes de concluir que no hay solución.

## 2.8. Tiempos de ejecución

Para evaluar si los tiempos de ejecución se corresponden con la complejidad teórica  $O(k \cdot n^3)$ , se diseñó un experimento con parámetros fijos  $k = 2$ ,  $b = 3$ ,  $D = 40$  y tamaños de entrada crecientes  $n \in \{5, 10, 20, 30, 50, 75, 100, 150, 200\}$ . Para cada valor de  $n$  se generaron 7 instancias aleatorias con semillas distintas, se descartaron el mínimo y el máximo, y se promedió el tiempo de las 5 mediciones restantes. Con  $k$  fijo, la complejidad predicha es  $O(n^3)$ ; para contrastar, se ajustan simultáneamente una curva cúbica  $f(n) = C_3 \cdot n^3$  y una cuadrática  $f(n) = C_2 \cdot n^2$ , ambas por mínimos cuadrados.

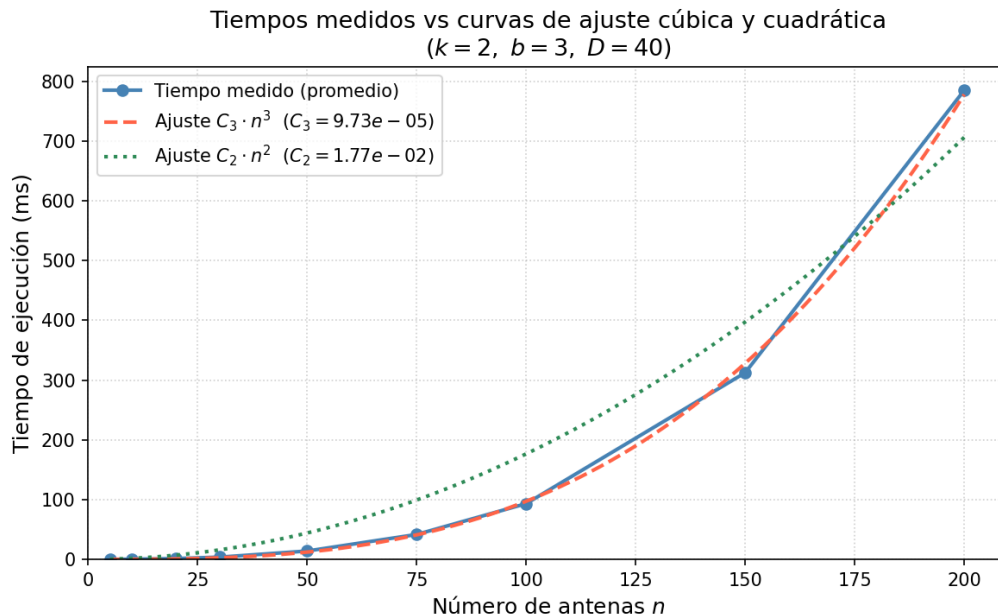


Figura 2: Tiempos de ejecución medidos (azul) frente al ajuste cúbico  $C_3 \cdot n^3$  (rojo discontinuo) y el ajuste cuadrático  $C_2 \cdot n^2$  (verde punteado), para  $k = 2$ ,  $b = 3$ ,  $D = 40$ . Generado con `analisis/tiempos.ipynb`.

**Análisis del ajuste.** La curva medida se ajusta con mayor precisión a  $C_3 \cdot n^3$  que a  $C_2 \cdot n^2$ : para valores grandes de  $n$  la curva cuadrática subestima sistemáticamente los tiempos, mientras que la cúbica los sigue de cerca. Esto confirma la complejidad  $O(k \cdot n^3)$  determinada teóricamente. La leve desviación para  $n$  pequeños se debe a que el costo constante de inicialización de la red domina sobre el término cúbico. En instancias sin solución, Edmonds-Karp recorre igualmente toda la red residual antes de concluir, por lo que el comportamiento asintótico es el mismo.

**Efecto de los parámetros  $k$ ,  $b$  y  $D$ .** La Figura 3 muestra el tiempo en función de  $n$  al variar cada uno de los tres parámetros por separado.

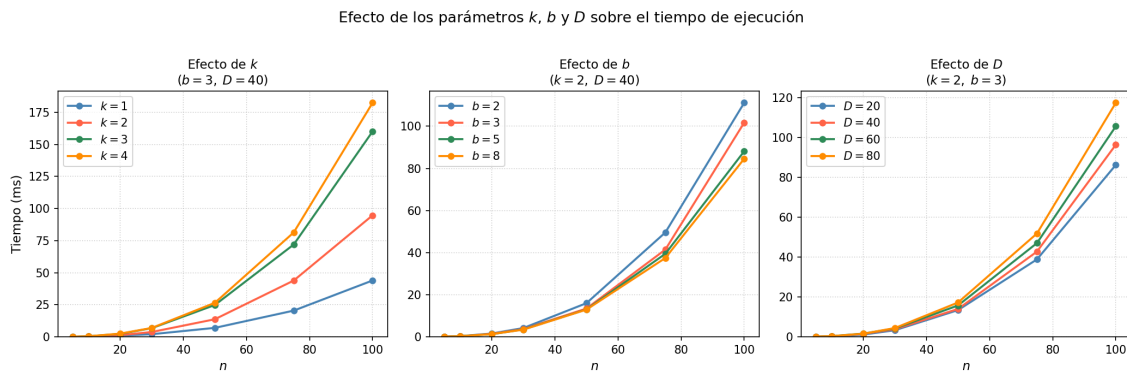


Figura 3: Efecto de  $k$ ,  $b$  y  $D$  sobre los tiempos de ejecución. Cada panel varía un parámetro manteniendo los demás fijos. Generado con `analisis/tiempos.ipynb`.

- **Efecto de  $k$ :** al aumentar  $k$ , las curvas se desplazan hacia arriba de forma aproximadamente proporcional, consistente con la dependencia lineal  $O(k \cdot n^3)$ . Un mayor  $k$  implica más caminos aumentantes necesarios (el flujo objetivo es  $n \cdot k$ ), por lo que Edmonds-Karp ejecuta más iteraciones.
- **Efecto de  $b$ :** valores mayores de  $b$  reducen el tiempo, ya que al relajar la restricción de capacidad en los nodos  $B_j \rightarrow T$  el algoritmo encuentra caminos aumentantes con menos rerouting. Para  $b$  muy grande, la restricción efectiva pasa a ser  $k$  (cada  $A_i$  solo puede enviar  $k$  unidades), y el tiempo converge.
- **Efecto de  $D$ :** un umbral de distancia mayor aumenta la densidad de aristas intermedias  $A_i \rightarrow B_j$ , lo que incrementa el tamaño de la red y el costo de cada BFS. Esto se refleja en tiempos ligeramente mayores a medida que  $D$  crece, especialmente para  $n$  grande.

La Figura 4 superpone todas las curvas anteriores en un único gráfico, distinguiendo cada parámetro por estilo de línea y paleta de color.

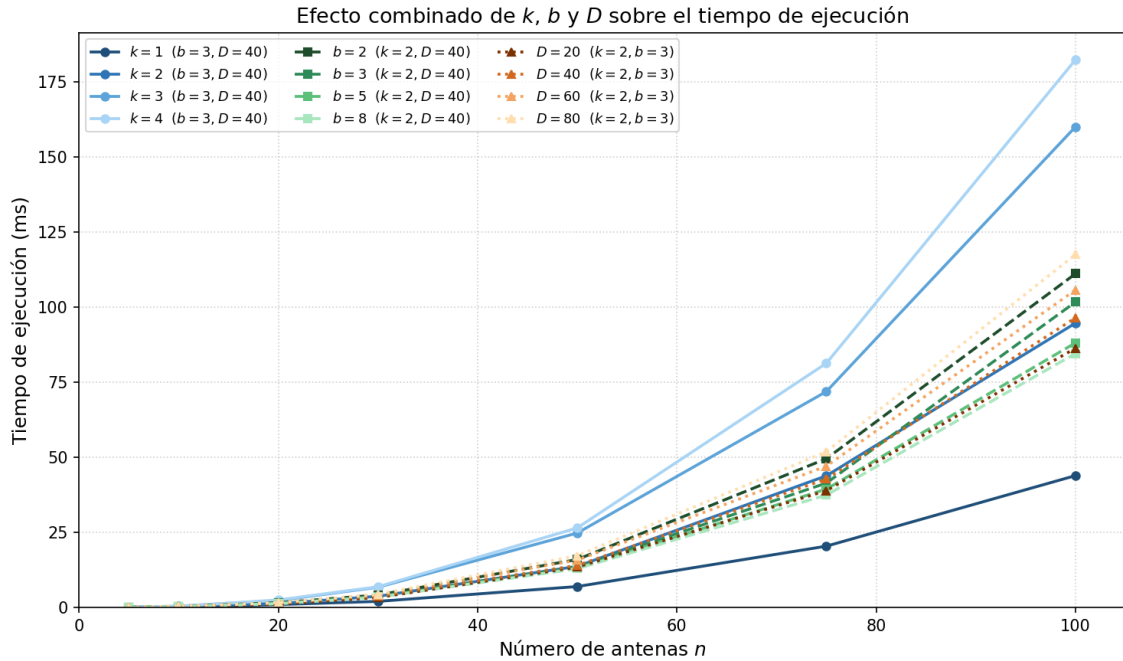


Figura 4: Efecto combinado de  $k$  (líneas sólidas, azules),  $b$  (líneas discontinuas, verdes) y  $D$  (líneas punteadas, naranjas) sobre el tiempo de ejecución en función de  $n$ . Es notorio el impacto de  $k$  en los tiempos de ejecución. Generado con `analisis/tiempos.ipynb`.

## 2.9. Seguimiento

**Instancia.** Seguimos la ejecución sobre `casos/caso1.toml`:  $n = 5$  antenas con parámetros  $k = 2$ ,  $b = 3$ ,  $D = 7$ . El flujo objetivo es  $n \cdot k = 5 \cdot 2 = 10$ . La traza completa se puede reproducir con:

```
python antenas.py --verbose casos/caso1.toml
```

### 2.9.1. Construcción de la red de flujo

Con  $D = 7$ , se agrega la arista  $A_i \rightarrow B_j$  ( $\text{cap} = 1$ ) para cada par de antenas distintas con  $d[i][j] < 7$ . Las aristas  $S \rightarrow A_i$  tienen capacidad  $k = 2$  y las  $B_j \rightarrow T$  tienen capacidad  $b = 3$ .

| Antena $i$ | Vecinos $j$ con $d[i][j] < 7 \Rightarrow$ aristas $A_i \rightarrow B_j$    |
|------------|----------------------------------------------------------------------------|
| 1          | $j = 2$ ( $d=5$ ), $j = 3$ ( $d=6$ )                                       |
| 2          | $j = 1$ ( $d=5$ ), $j = 3$ ( $d=1$ ), $j = 4$ ( $d=2$ ), $j = 5$ ( $d=3$ ) |
| 3          | $j = 1$ ( $d=3$ ), $j = 2$ ( $d=1$ ), $j = 4$ ( $d=1$ ), $j = 5$ ( $d=2$ ) |
| 4          | $j = 2$ ( $d=2$ ), $j = 3$ ( $d=1$ ), $j = 5$ ( $d=1$ )                    |
| 5          | $j = 2$ ( $d=6$ ), $j = 3$ ( $d=2$ ), $j = 4$ ( $d=1$ )                    |

Cuadro 6: Vecinos de cada antena y aristas intermedias generadas (todas con  $\text{cap} = 1$ ).

La red resultante tiene  $2n + 2 = 12$  nodos y  $n + 20 + n = 30$  aristas (5 desde  $S$ , 20 intermedias, 5 hacia  $T$ ).

### 2.9.2. Iteraciones de Edmonds-Karp

Cada iteración ejecuta un BFS sobre la red residual para hallar el camino más corto desde  $S$  hasta  $T$ , y aumenta el flujo por la capacidad del cuello de botella.

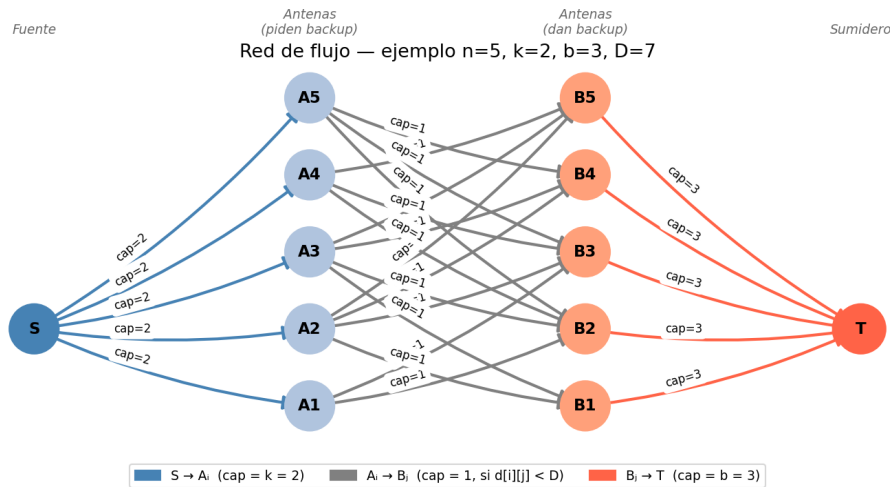


Figura 5: Red de flujo construida para el caso 1. Generado con `grafico_red.py`.

| Iter. | Camino aumentante                                                                   | Cuello | Flujo acum. |
|-------|-------------------------------------------------------------------------------------|--------|-------------|
| 1     | $S \rightarrow A_1 \rightarrow B_2 \rightarrow T$                                   | 1      | 1           |
| 2     | $S \rightarrow A_1 \rightarrow B_3 \rightarrow T$                                   | 1      | 2           |
| 3     | $S \rightarrow A_2 \rightarrow B_1 \rightarrow T$                                   | 1      | 3           |
| 4     | $S \rightarrow A_2 \rightarrow B_3 \rightarrow T$                                   | 1      | 4           |
| 5     | $S \rightarrow A_3 \rightarrow B_1 \rightarrow T$                                   | 1      | 5           |
| 6     | $S \rightarrow A_3 \rightarrow B_2 \rightarrow T$                                   | 1      | 6           |
| 7     | $S \rightarrow A_4 \rightarrow B_2 \rightarrow T$                                   | 1      | 7           |
| 8     | $S \rightarrow A_4 \rightarrow B_3 \rightarrow T$                                   | 1      | 8           |
| 9     | $S \rightarrow A_5 \rightarrow B_4 \rightarrow T$                                   | 1      | 9           |
| 10    | $S \rightarrow A_5 \rightarrow B_2 \rightarrow A_3 \rightarrow B_4 \rightarrow T$ * | 1      | 10          |

Cuadro 7: Caminos aumentantes encontrados por Edmonds-Karp. \*La iteración 10 usa la arista residual  $B_2 \rightarrow A_3$ .

**Nota sobre la iteración 10.** Tras la iteración 9,  $A_5$  aún necesita 1 unidad de flujo pero sus caminos directos están bloqueados:  $B_2 \rightarrow T$  y  $B_3 \rightarrow T$  están saturadas (cada una absorbió 3 unidades en las iteraciones 1,6,7 y 2,4,8 respectivamente), y  $A_5 \rightarrow B_4$  fue usada en la iteración 9. El BFS encuentra entonces un camino de largo 5 que atraviesa la arista residual  $B_2 \rightarrow A_3$  (el reverso de la arista  $A_3 \rightarrow B_2$  saturada en la iteración 6). Esto cancela la asignación  $3 \in \text{backup}(3)$  y la reemplaza por  $4 \in \text{backup}(3)$ , liberando a  $B_2$  para que  $A_5$  lo utilice. Este es el mecanismo central de Edmonds-Karp: la red residual permite “deshacer” decisiones anteriores en busca de un mejor flujo global.

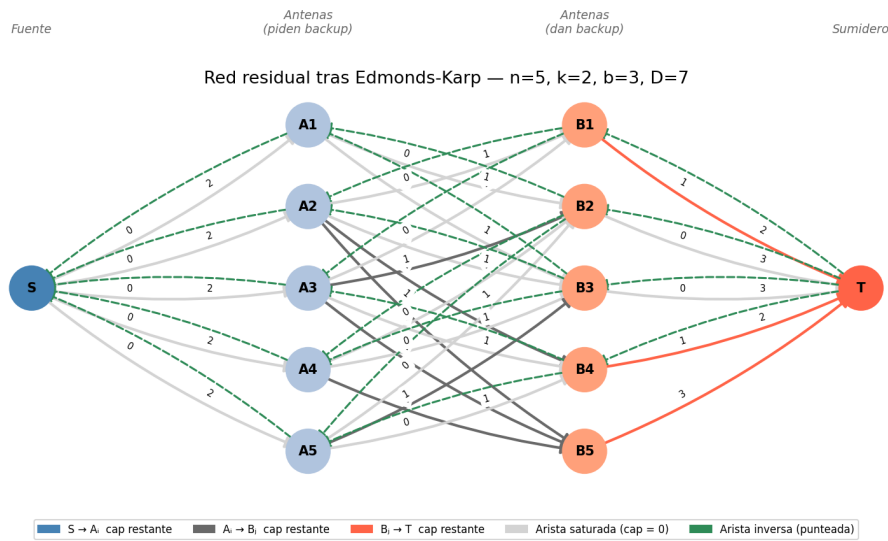


Figura 6: Red de flujo residual para el caso 1 señalando las aristas saturadas, las aristas con capacidad residual y los caminos. Generado con `grafico_red.py`.

### 2.9.3. Verificación y reconstrucción

El flujo total alcanzado es  $10 = n \cdot k$ , por lo que **existe solución**. Los conjuntos de backup se leen de las aristas saturadas (capacidad residual = 0) en la red final:

| Antena $i$ | Aristas saturadas $A_i \rightarrow B_j$    | Backup     | Apariciones como backup |
|------------|--------------------------------------------|------------|-------------------------|
| 1          | $A_1 \rightarrow B_2, A_1 \rightarrow B_3$ | $\{2, 3\}$ | 2 (antenas 4, 5)        |
| 2          | $A_2 \rightarrow B_1, A_2 \rightarrow B_3$ | $\{1, 3\}$ | 3 (antenas 1, 4, 5)     |
| 3          | $A_3 \rightarrow B_1, A_3 \rightarrow B_4$ | $\{1, 4\}$ | 3 (antenas 1, 2, 4)     |
| 4          | $A_4 \rightarrow B_2, A_4 \rightarrow B_3$ | $\{2, 3\}$ | 2 (antenas 3, 5)        |
| 5          | $A_5 \rightarrow B_2, A_5 \rightarrow B_4$ | $\{2, 4\}$ | 0                       |

Cuadro 8: Reconstrucción final. Cada antena obtiene exactamente  $k = 2$  backups y ninguna aparece en más de  $b = 3$  conjuntos.

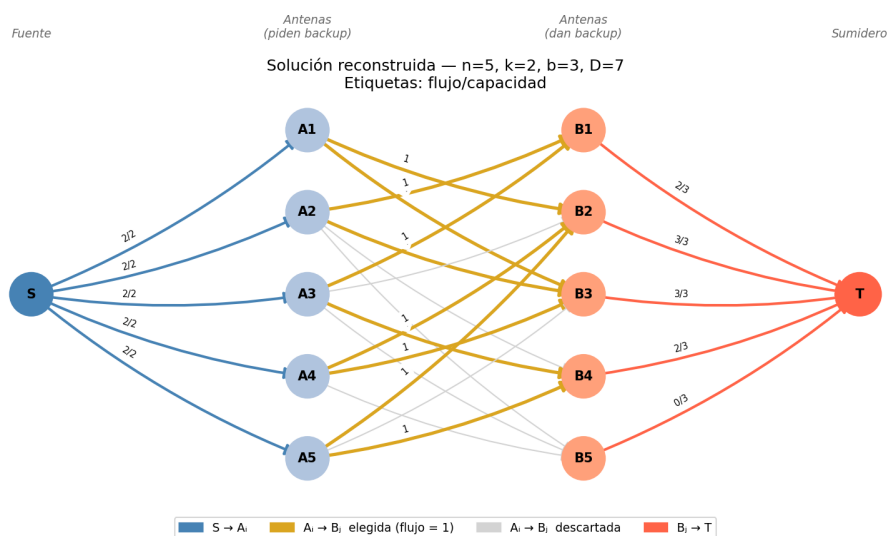


Figura 7: Aristas utilizadas en la reconstrucción de la solución. Generado con `grafico_red.py`.



### 3. Problema 3 - Algoritmo de Aproximación

#### 3.1. Descripción del problema

Tenemos un conjunto de enteros positivos  $A = \{a_1, a_2, \dots, a_n\}$  y otro entero positivo  $B$ . Llamaremos subconjunto factible a un subconjunto  $S$  de  $A$  tal que la suma de sus elementos sea a lo sumo el valor  $B$ .

Queremos encontrar un subconjunto factible  $S$  de  $A$  tal que la suma de sus elementos sea la mayor posible. Por ejemplo, si  $A = \{9, 3, 5\}$  y  $B = 13$ , entonces la solución óptima es  $S = \{9, 3\}$ .

El siguiente es un algoritmo para encontrar un subconjunto factible (no necesariamente el óptimo):

```
1 subconjunto_factible(A, B)
2   S = {}
3   T = 0
4   Para i = 1, 2, ..., n
5       Si T + A[i] <= B, entonces:
6           S = S ∪ A[i]
7           T = T + A[i]
8       fin si
9   fin para
10  devolver S
```

#### 3.2. Conjunto Factible de S Menor a la Mitad del Óptimo

El algoritmo dado posee varios problemas. Para un conjunto  $A = \{10, 20, 80\}$  y el número entero positivo  $B = 105$ , dará un conjunto factible  $S = \{10, 20\}$  donde la suma de sus elementos da un total de 30. Y el óptimo en realidad sería  $S = \{20, 80\}$ , que da un total de 100. Podemos concluir que el conjunto factible de  $S$  para dichas instancias en el algoritmo sería menor a la mitad del conjunto factible de  $S$  óptimo.

#### 3.3. Propuesta de Otro Algoritmo

Hemos propuesto otro algoritmo para obtener una garantía donde la suma del conjunto factible  $S$  sea al menos la mitad que cualquier otro conjunto factible de  $A$ :

```
1 subconjunto_factible(A, B)
2   ordenar de mayor a menor a A
3   S = {}
4   T = 0
5   Para i = 1, 2, ..., n
6       Si T + A[i] <= B, entonces:
7           S = S ∪ A[i]
8           T = T + A[i]
9       fin si
10  fin para
11  devolver S
```

Nos quedamos con el algoritmo dado pero antes de ejecutar dicho algoritmo ordenamos a  $A$  de mayor a menor. Con dicho algoritmo no va a pasar lo que pasaba en el apartado anterior ya que se obtendría  $S = \{20, 80\}$ .

Si analizamos el seguimiento del algoritmo con  $A = [51, 50, 50]$  y  $B = 100$  podemos ver lo siguiente:

Paso 1. Se ordena la lista  $A$  de mayor a menor. Quedaría de la misma forma que se recibió dado que está ordenado de esa forma

Paso 2. Se itera la lista ordenada de A:

En  $i = 1$ ,  $A[i] = 51$  y  $T = 0$ . Por lo que  $T + A[i] \leq B$ . Entonces  $S = [51]$  y  $T = 51$

En  $i = 2$ ,  $A[i] = 50$ . En este caso no se cumple la condición de que  $T + A[i] \leq B$ .

En  $i = 3$ ,  $A[i] = 50$ . Pasaría lo mismo que en  $i = 2$

Paso 3: se devuelve  $S = [51]$

Se devuelve un conjunto factible  $S = [51]$  pero el óptimo debería ser  $S = [50, 50]$ . Con esto podemos decir que como garantía  $T^* \geq OPT/2$  donde  $T^*$  es el  $T$  obtenido al final del algoritmo, en este caso 51. Y  $OPT$  es el  $T$  que se obtendría en un resultado óptimo, en este caso 100

Las estructuras de datos utilizada para este algoritmo fue una lista que se usó para representar al conjunto de enteros positivos de A y al conjunto factible S.

### 3.4. Análisis de Complejidad

Según el seguimiento del código que se hizo en el apartado anterior, se puede observar que la complejidad temporal sería  $O(n \log n)$  en el peor caso. Siendo  $n$  el conjunto (lista) de números enteros positivos que pertenecen a A.

Primero se ordena la lista A con una complejidad de  $O(n \log n)$ . Luego, se recorren los números enteros positivos de dicha lista haciendo la comparación de que  $T + A[i]$  sea menor o igual a B, insertando el número en S e incrementando el número de T a  $A[i]$ . Eso llevaría a una complejidad de  $O(n)$  de la iteración, Y, como  $O(n)$  es menor que  $O(n \log n)$ , la complejidad temporal sería  $O(n \log n)$ . Se puede ver en este gráfico que los resultados obtenidos del algoritmo propuesto se aproximan a la curva teórica  $O(n \log n)$ :

| Total de Elementos en A | Tiempo de Ejecución (en ms) |
|-------------------------|-----------------------------|
| 3                       | 0,00912 ms                  |
| 11                      | 0,0117 ms                   |
| 26                      | 0,0159 ms                   |
| 58                      | 0,0297 ms                   |
| 89                      | 0,0407 ms                   |
| 104                     | 0,05 ms                     |

Cuadro 9: Tiempos de ejecución de cada resultado

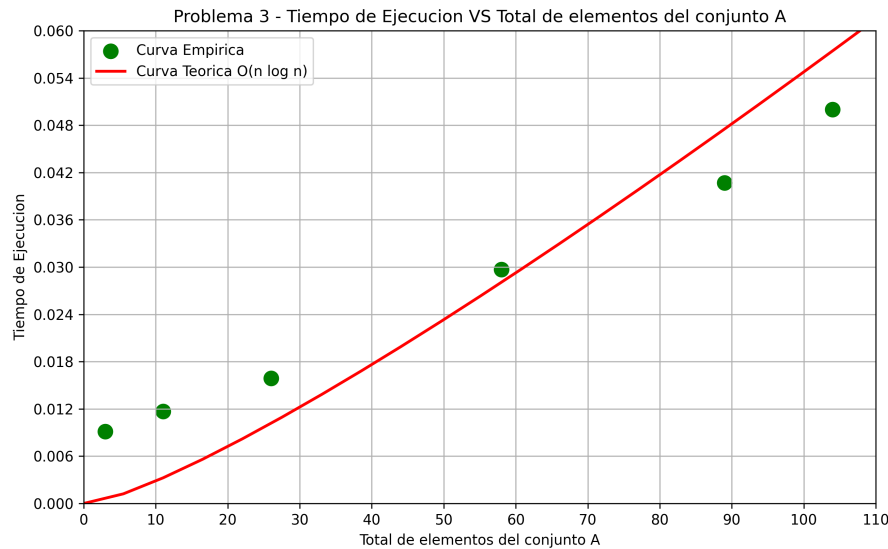


Figura 8: Gráfico de los tiempos de ejecución de cada resultado

En cuanto la complejidad espacial, se usa una lista  $S$  para almacenar los números enteros positivos que cumplieron con el conjunto factible. En el peor caso eso es  $O(n)$ , ya que en el peor caso todos los números de la lista  $A$  estarían dentro de la lista  $S$ .

### 3.5. Premisas

En este algoritmo propuesto las premisas a considerar serían:

1. Las operaciones aritméticas, la suma, la comparación y la acción de añadir un elemento a una lista tienen una complejidad temporal  $O(1)$ .
2. El ordenamiento de  $A$  tiene una complejidad temporal de  $O(n \log n)$ .

### 3.6. Supuestos

Para que el algoritmo propuesto sea realmente efectivo. Hay que tener en cuenta varios supuestos:

1. Los números del conjunto  $A$  no pueden ser negativos. Debido a que no solo pierde la garantía aproximada sino que también puede perder la correctitud básica. Por ejemplo, si tenemos  $A = [-1, -2, -3]$  y  $B = 10$  entonces  $S = [-1]$  con  $T = -1$  pero en realidad  $T$  debería ser 0.
2.  $B$  no puede ser negativo. Debido a que se pierde la correctitud básica. Por ejemplo, con  $A = [1, 2, 3]$  y  $B = -2$  entonces  $S = []$  y  $T = 0$ . Pero en realidad  $T$  debería ser como mucho  $-2$  y eso devuelve un conjunto infactible.

### 3.7. Limitaciones

Además de dichos supuestos, el algoritmo propuesto consta de algunas limitaciones:

1. Complejidad temporal mayor que el anterior algoritmo. Si bien en el algoritmo propuesto se llegó a una garantía aproximada mejor que el anterior algoritmo. Concluimos que el algoritmo

propuesto tiene una complejidad temporal de  $O(n \log n)$  pero el algoritmo anterior tenía una complejidad temporal de  $O(n)$  debido a que no ordena el conjunto  $A$ . Por lo que el algoritmo propuesto es más complejo a nivel temporal que el algoritmo anterior.

2. No garantiza la optimalidad. Existen instancias donde el algoritmo propuesto no alcanza al óptimo. Por ejemplo en  $A = \{51, 50, 50\}$  y  $B = 100$ , en donde  $S = 51$  y  $T = 51$ . Pero el óptimo de la solución de dichas instancias debería ser  $S^* = \{50, 50\}$  y  $T^* = 100$ .

### 3.8. Conclusión

En muchos problemas de optimización no siempre es necesario encontrar la solución óptima global, sino que puede ser más importante obtener una solución suficientemente buena en un tiempo razonable. En este sentido, los algoritmos de aproximación permiten sacrificar optimalidad a cambio de eficiencia computacional.

Como se observó en este trabajo, es posible mejorar la garantía de calidad de la solución obtenida, pero esto puede implicar un aumento en la complejidad temporal del algoritmo. Por lo tanto, existe un compromiso entre la calidad de la solución y el costo computacional, y la elección del algoritmo dependerá del contexto y de las restricciones del problema.

## 4. Problema 4: Algoritmos aleatorios

### 4.1. Descripción del problema

El problema de 3-Coloreo es un problema de decisión: dado un grafo  $G = (V, E)$ , ¿existe una forma de colorear todos los vértices usando a lo sumo 3 colores, tal que cada par de vértices vecinos tenga diferentes colores? Sin embargo, también posee su contraparte como problema de optimización. Dado un grafo  $G = (V, E)$ , se quiere colorear cada vértice con uno de tres colores, incluso si no se pudiera colorear con colores diferentes cada par de vértices vecinos. Una arista  $e = (u, v)$  será satisfecha si los colores asignados a  $u$  y  $v$  son diferentes.

Se considera un 3-Coloreo que maximiza la cantidad de aristas satisfechas, siendo  $c^*$  esta cantidad. Se busca un algoritmo aleatorio de tiempo polinomial que determine un 3-Coloreo cuyo número esperado de aristas satisfechas sea al menos  $\frac{2}{3}c^*$  aristas.

### 4.2. Algoritmo diseñado

El problema fue resuelto implementando un algoritmo que, dado un grafo cualquiera, le asigna un color aleatorio (de tres colores posibles) a cada uno de sus vértices.

```
1 funcion COLOR_ALEATORIO()  
2     retornar numero_aleatorio(1, 3)  
3 fin funcion  
4  
5 funcion COLOREAR_GRAFO(grafo)  
6     vertices <- obtener_vertices(grafo)  
7     vertices_coloreados <- {}  
8  
9     para cada vertice en vertices hacer  
10        vertices_coloreados[vertice] <- COLOR_ALEATORIO()  
11     fin para  
12  
13     retornar vertices_coloreados  
14 fin funcion
```

La probabilidad con la que se le asigna un color  $i$  a un vértice  $v$  es  $\frac{1}{3}$ .

Para implementar el algoritmo se hizo uso del TDA Grafo y de un diccionario para guardar el estado de cada uno de los vértices a medida que se iban coloreando.

En lo que respecta a la complejidad de la solución implementada, la misma es  $O(|V|)$ , siendo  $|V|$  la cantidad de vértices del grafo. El algoritmo recorre la lista de vértices linealmente y le asigna un color a cada uno de forma aleatoria e independiente, lo cual puede considerarse como una operación de costo constante.

### 4.3. Demostración del valor mínimo esperado

Dado un grafo  $G = (V, E)$ , se quiere obtener un 3-coloreo aleatorio del mismo cuyo número esperado de aristas satisfechas sea  $\frac{2}{3}c^*$ .

1. Una arista  $e = (u, v)$  queda satisfecha si  $\text{color}(u) \neq \text{color}(v)$ .
2. Un vértice  $v$  puede colorearse con uno de los tres colores disponibles, teniendo cada uno una probabilidad de  $\frac{1}{3}$  de ser escogido.
3. Dada una arista  $e = (u, v)$ , tanto  $u$  como  $v$  pueden colorearse de tres colores posibles (1, 2 o 3). Si se le asigna un color aleatorio a cada vértice del grafo sin hacer ningún tipo de distinción, entonces los eventos de asignación de color para  $u$  y  $v$  son completamente independientes.
4. La probabilidad de que  $u$  y  $v$  tengan el mismo color es  $\frac{1}{3}$ . Esto se puede calcular evaluando la probabilidad de que ambos vértices compartan el color 1, el color 2 o el color 3. Como

cada uno de estos eventos son mutuamente excluyentes (no pueden ocurrir a la vez), la probabilidad se define como la suma de sus probabilidades individuales:

$$P(\text{mismo color}) = P(u = 1 \wedge v = 1) + P(u = 2 \wedge v = 2) + P(u = 3 \wedge v = 3)$$

Dado que la elección de color de  $u$  y de  $v$  son eventos independientes, la probabilidad de la intersección es el producto de las probabilidades:

$$P(\text{mismo color}) = P(u = 1)P(v = 1) + P(u = 2)P(v = 2) + P(u = 3)P(v = 3)$$

$$P(\text{mismo color}) = \left(\frac{1}{3}\right)\left(\frac{1}{3}\right) + \left(\frac{1}{3}\right)\left(\frac{1}{3}\right) + \left(\frac{1}{3}\right)\left(\frac{1}{3}\right) = 3 \cdot \frac{1}{9} = \frac{1}{3}$$

5. Como la probabilidad de que los vértices de una arista tengan el mismo color es  $\frac{1}{3}$ , la probabilidad de que una arista no sea satisfecha es  $\frac{2}{3}$ . Por la regla del complemento, la probabilidad de que la arista **sí** sea satisfecha es  $1 - \frac{2}{3} = \frac{1}{3}$ .
6. Sea  $X_e \in \{0, 1\}$  una variable aleatoria discreta indicadora que representa si la arista  $e$  fue satisfecha o no (1 si fue satisfecha, 0 si no). Por el punto anterior, sabemos que  $X_e = 1$  con probabilidad  $\frac{1}{3}$ , y  $X_e = 0$  con probabilidad  $\frac{2}{3}$ .
7. La esperanza matemática de  $X_e$  se calcula como:

$$E[X_e] = 0 \cdot P(X_e = 0) + 1 \cdot P(X_e = 1)$$

$$E[X_e] = 0 \cdot \left(\frac{2}{3}\right) + 1 \cdot \left(\frac{1}{3}\right) = \frac{1}{3}$$

8. La cantidad total de aristas satisfechas es la suma de las variables indicadoras de todas las aristas:

$$X = \sum_{e \in E} X_e$$

Como cada  $X_e$  es una variable aleatoria,  $X$  también lo es. Aplicando la propiedad de la linealidad de la esperanza:

$$E[X] = E\left[\sum_{e \in E} X_e\right] = \sum_{e \in E} E[X_e]$$

$$E[X] = \sum_{e \in E} \frac{1}{3} = \frac{1}{3}|E|$$

Donde  $|E|$  es la cantidad total de aristas del grafo.

9. Sea  $c^*$  la cantidad máxima de aristas satisfechas proveniente del problema de optimización. En todo momento se cumple que la solución óptima no puede superar la cantidad total de aristas del grafo, es decir,  $c^* \leq |E|$ . Multiplicando ambos lados por  $\frac{1}{3}$ :

$$\frac{1}{3}c^* \leq \frac{1}{3}|E|$$

Reemplazando en el resultado obtenido en el punto 8:

$$E[X] = \frac{1}{3}|E| \geq \frac{1}{3}c^*$$

$$E[X] \geq \frac{1}{3}c^*$$

Lo cual demuestra que si se colorean todos los vértices del grafo eligiendo para cada uno un color aleatorio (de manera independiente y uniforme entre tres posibles colores), la cantidad esperada de aristas satisfechas es al menos  $\frac{1}{3}$  del óptimo  $c^*$ .

#### 4.4. Seguimiento del Algoritmo

En esta sección se analiza el comportamiento práctico del algoritmo diseñado. Es fundamental destacar que la solución implementada constituye un **algoritmo de Monte Carlo**. Esto se debe a que el tiempo de ejecución es determinísticamente polinomial ( $O(|V|)$ ), pero el resultado es probabilístico: el algoritmo siempre termina en tiempo acotado, pero existe una probabilidad de que la solución obtenida no alcance el valor óptimo o incluso el valor esperado en una ejecución única.

Como se demostró anteriormente, la probabilidad de que una arista individual no sea satisfecha es de  $1/3$ . Por lo tanto, aunque el algoritmo es eficiente, su éxito en cumplir la cota de  $\frac{2}{3}c^*$  depende de la distribución aleatoria en cada ejecución.

##### 4.4.1. Ejemplo de Seguimiento: Grafo Lineal

Para el seguimiento, utilizaremos un grafo lineal  $G = (V, E)$  con 6 vértices y 5 aristas:

$$A - B - C - D - E - F$$

El número máximo de aristas satisfechas es  $c^* = |E| = 5$ . El objetivo esperado es satisfacer al menos:

$$\frac{2}{3} \cdot 5 = 3,33 \implies \text{Al menos 4 aristas satisfechas.}$$

**Caso 1: Ejecución Exitosa** ( $X \geq \frac{2}{3}c^*$ ) En este caso, el generador de números aleatorios asigna colores que permiten una alta tasa de satisfacción.

| Paso | Vértice | Color Asignado | Estado de Arista                |
|------|---------|----------------|---------------------------------|
| 1    | A       | 1              | -                               |
| 2    | B       | 2              | (A,B) Satisfecha ( $1 \neq 2$ ) |
| 3    | C       | 1              | (B,C) Satisfecha ( $2 \neq 1$ ) |
| 4    | D       | 3              | (C,D) Satisfecha ( $1 \neq 3$ ) |
| 5    | E       | 2              | (D,E) Satisfecha ( $3 \neq 2$ ) |
| 6    | F       | 1              | (E,F) Satisfecha ( $2 \neq 1$ ) |

**Resultado:** 5 aristas satisfechas. Como  $5 \geq 3,33$ , se cumple la condición de aproximación.

**Caso 2: Ejecución Insatisfactoria** ( $X < \frac{2}{3}c^*$ ) En este caso, la aleatoriedad produce colisiones de color en los nodos vecinos, fallando en superar el promedio esperado.

| Paso | Vértice | Color Asignado | Estado de Arista                       |
|------|---------|----------------|----------------------------------------|
| 1    | A       | 1              | -                                      |
| 2    | B       | 1              | (A,B) <b>No Satisfecha</b> ( $1 = 1$ ) |
| 3    | C       | 2              | (B,C) Satisfecha ( $1 \neq 2$ )        |
| 4    | D       | 2              | (C,D) <b>No Satisfecha</b> ( $2 = 2$ ) |
| 5    | E       | 3              | (D,E) Satisfecha ( $2 \neq 3$ )        |
| 6    | F       | 3              | (E,F) <b>No Satisfecha</b> ( $3 = 3$ ) |

**Resultado:** 2 aristas satisfechas. Como  $2 < 3,33$ , esta ejecución particular no cumple con la cota esperada, ilustrando la probabilidad de fallo inherente a los algoritmos de Monte Carlo.

#### 4.5. Informe de Resultados

Como se demostró en las secciones anteriores, el enfoque propuesto constituye un algoritmo probabilístico de tipo Monte Carlo. Esto implica que, aunque el tiempo de ejecución está garantizado, existe una probabilidad inherente de que en una ejecución aislada no se alcance la cota mínima esperada de  $\frac{2}{3}c^*$  aristas satisfechas.

Para mitigar este comportamiento y garantizar una solución robusta sin descartar resultados potencialmente válidos, se implementó una **estrategia de reintentos**. El algoritmo evalúa la cantidad de aristas satisfechas generadas; si no se cumple la condición de los  $2/3$ , se vuelve a ejecutar la asignación aleatoria. Se estableció un límite máximo de 10 reintentos (un parámetro que puede ser ajustado según las necesidades del problema). Esta estrategia no solo asegura empíricamente el cumplimiento de la cota, sino que también sirve para demostrar la naturaleza netamente aleatoria del algoritmo: ante un resultado no deseado, una nueva iteración genera una distribución completamente diferente.

#### 4.5.1. Grafos de Prueba

Para la evaluación empírica de la solución, se seleccionaron tres grafos diferentes, los cuales permiten observar el comportamiento del coloreo aleatorio bajo distintas densidades y estructuras:

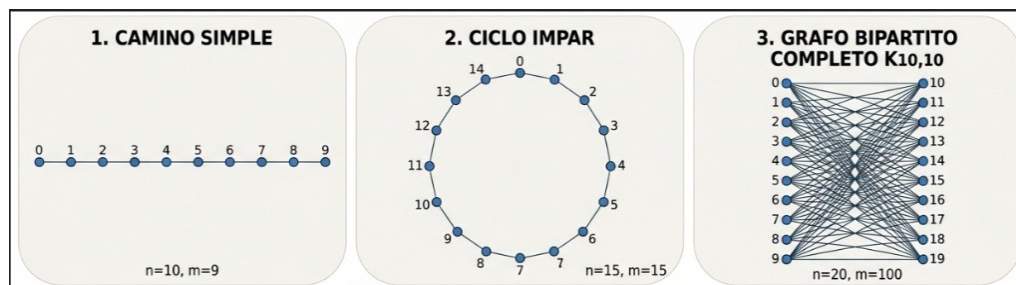


Figura 9: Grafos utilizados para los casos de prueba: (1) Camino Simple, (2) Ciclo Impar y (3) Grafo Bipartito Completo  $K_{10,10}$ .

#### 4.5.2. Resultados por Grafo

A continuación se presentan los resultados obtenidos tras ejecutar el algoritmo sobre cada uno de los grafos ilustrados. Para cada caso se detalla el coloreo final asignado a cada vértice y las métricas de éxito. Cabe destacar que en la métrica de Reintentos el 0 indica que el algoritmo logró la cota esperada en su primer intento.

**Grafo 1: Camino Simple** ( $n = 10, m = 9$ ) Este caso representa la estructura más básica, un grafo lineal sin ciclos. En esta ejecución, el algoritmo requirió de 1 reintento para superar la cota de 6 aristas ( $2/3$  de 9), logrando un total de 8 aristas satisfechas.

| Métrica                        | Valor |
|--------------------------------|-------|
| Total de aristas               | 9     |
| Aristas satisfechas            | 8     |
| Aristas no satisfechas         | 1     |
| Cumple $\geq 2/3$ aristas      | Sí    |
| Reintentos hasta salida válida | 1     |

Cuadro 10: Métricas de ejecución para el Camino Simple.

| V | Color | V | Color | V | Color | V | Color | V | Color |
|---|-------|---|-------|---|-------|---|-------|---|-------|
| 0 | 2     | 2 | 2     | 4 | 2     | 6 | 3     | 8 | 3     |
| 1 | 3     | 3 | 1     | 5 | 3     | 7 | 2     | 9 | 1     |

Cuadro 11: Coloreo resultante para el Grafo 1.



**Grafo 2: Ciclo Impar** ( $n = 15, m = 15$ ) Un ciclo de longitud impar es un caso clásico donde el 2-coloreo perfecto es imposible. Utilizando el enfoque de 3-coloreo aleatorio, el algoritmo alcanzó 10 aristas satisfechas (el mínimo exacto de  $2/3$  de 15) en su primer intento, sin necesidad de repeticiones.

| Métrica                        | Valor |
|--------------------------------|-------|
| Total de aristas               | 15    |
| Aristas satisfechas            | 10    |
| Aristas no satisfechas         | 5     |
| Cumple $\geq 2/3$ aristas      | Sí    |
| Reintentos hasta salida válida | 0     |

Cuadro 12: Métricas de ejecución para el Ciclo Impar.

| V | Color | V | Color | V | Color | V  | Color | V  | Color |
|---|-------|---|-------|---|-------|----|-------|----|-------|
| 0 | 1     | 3 | 3     | 6 | 3     | 9  | 1     | 12 | 2     |
| 1 | 1     | 4 | 3     | 7 | 3     | 10 | 2     | 13 | 3     |
| 2 | 2     | 5 | 2     | 8 | 1     | 11 | 3     | 14 | 3     |

Cuadro 13: Coloreo resultante para el Grafo 2.

**Grafo 3: Grafo Bipartito Completo**  $K_{10,10}$  ( $n = 20, m = 100$ ) Este grafo presenta una densidad de aristas mucho mayor al conectar todos los nodos de un conjunto con el otro. La cota requerida es de al menos 66.6 aristas. El algoritmo logró superar holgadamente esta cota alcanzando 70 aristas satisfechas en el primer intento.

| Métrica                        | Valor |
|--------------------------------|-------|
| Total de aristas               | 100   |
| Aristas satisfechas            | 70    |
| Aristas no satisfechas         | 30    |
| Cumple $\geq 2/3$ aristas      | Sí    |
| Reintentos hasta salida válida | 0     |

Cuadro 14: Métricas de ejecución para el Grafo Bipartito Completo  $K_{10,10}$ .

| V | Color | V | Color | V  | Color | V  | Color | V  | Color |
|---|-------|---|-------|----|-------|----|-------|----|-------|
| 0 | 2     | 4 | 2     | 8  | 1     | 12 | 3     | 16 | 3     |
| 1 | 1     | 5 | 2     | 9  | 1     | 13 | 2     | 17 | 3     |
| 2 | 1     | 6 | 2     | 10 | 2     | 14 | 2     | 18 | 2     |
| 3 | 3     | 7 | 3     | 11 | 3     | 15 | 3     | 19 | 2     |

Cuadro 15: Coloreo resultante para el Grafo 3.